Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231
www.wabtec.com

Vapor Stone Rail Systems

Generic Software Documentation Project

Project # 7260

Software Design Description (SDD)
Real-Time Data Monitoring

Contract Number		CDRL Number	
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	Thomas Marmeth Software Designer	2011-09-29 On behalf of Martin Ouellet
Verified :	Martin Ouellet, Eng. Software Department Supervisor	2011-09-30
Approved :	David Beaudoin, Eng. Software and Electrical Eng. Manager	2011-09-29

Document No.	**201111585**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-09-29**

Based on SDD Template S17-07 Rev.1

Revision List

Rev.	ECD#	Date	By	Description of changes
1	---	2011-09-29	Thomas Marmeth	Released version.
0.1	---	2011-09-28	Thomas Marmeth	Implemented comments from inspection I-1811.
0	---	2011-09-01	Thomas Marmeth	Original issue

Preface

This Software Design Description (SDD) complies with the requirements of the IEEE 1016-1998 and IEEE Std 1558-2005, IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems approved on 2005-03-23.

Table of Contents

Revision List 2
Preface 3
Table of Contents 4
List of Figures 5
List of Tables 5

1 INTRODUCTION 6

1.1.1 Purpose 6
1.1.2 Scope 6
1.1.3 Definitions and Acronyms 6
1.1.4 Acronyms 7
1.1.5 Document Organization 7

2 REFERENCES 9

3 SOFTWARE DESIGN 10

3.1 Decomposition Description 10
3.1.1 RTDM Module Description 10
3.1.2 Concurrent Process Decomposition 10
3.1.3 Data Decomposition 11
3.2 Dependency Description 13
3.2.1 Inter-Module Dependencies 13
3.2.2 Inter-Process Dependencies 13
3.2.3 Data Dependencies 14
3.3 Interface Description 15
3.3.1 RTDM functions 15
3.4 Detailed Design 18
3.4.1 RTDM Module Design 18
3.4.2 LDTS_initializeFT 20
3.4.3 LDTS_FileTransfer 20
3.4.4 LDTS_Recording 21
3.4.5 LDTS_ResetEvent 26
3.4.6 LDTS_SnapshotDelete 27
3.4.7 Data Entities Detailed Design 28

List of Figures

Figure 3-1: Inter-Module Dependencies Diagram 13
Figure 3-2: Inter-Process Dependencies Diagram 13
Figure 3-3: LDTS Check Snapshot 18
Figure 3-4: Directory Lookup 19
Figure 3-5: LDTS_initializeFT module 20
Figure 3-6: LDTS File Transfer 21
Figure 3-7: LDTS Recording 22
Figure 3-8: LDTS Reset Event 26
Figure 3-9: Snapshot Deletion 27
Figure 3-10: LDTS Task 28

List of Tables

Table 3-1: Structure of Snapshots Log 11
Table 3-2: Structure of Snapshots Counter 12
Table 3-3: Structure of Circular Buffer for Snapshot Purposes 23
Table 3-4: Signals List Example 24

1 INTRODUCTION

This document is the Software Design Description of the Real-Time Data Monitoring (RTDM) Module developed by Vapor Stone Rail Systems.

The Real-Time Data Monitoring Module can be part of any of the Door Module System (DMS) Applications.

1.1.1 Purpose

The purpose of this Software Design Description (SDD) is to describe the software components and implementation details of the Real-Time Data Monitoring Module.

The intended audience is the Vapor Stone Rail Systems software development team and maintenance team members of the transit authority and the car builder.

1.1.2 Scope

The Real-Time Data Monitoring Module is a software module of the Door Module System (DMS) Application.

1.1.3 Definitions and Acronyms

1.1.3.1 Definitions

Boot Loader	Software starting the Door Module System application.
Car Network	Data bus and optionally trainlines connecting the Door Control Module with the rest of the train (others Door Control Modules in the car and others cars).
Communication Gateway Module	A specific processor based electronic module running a Communication Gateway Module Application. It acts as gateway between the Door Control Module Application and the Car Network/Portable Test Application.
Door Module System	A term that is used for generically identifying a processor based electronic module running any software within any Door Control unit.
Door Control Unit	The final unit that is used to control one or more door panels. This unit is normally made of one or two door control modules running the Door Control Module Application and possibly one Communication Gateway Module Application.
Door Control Module	A specific processor based electronic module running a Door Control Module Application. It is used for monitoring and controlling the door.
HW_COM	Communication interface used for communication between Portable Test Application and Door Control Module application, that can be USB, serial or Ethernet.

1.1.3.2 Event Level Definitions

Event:	Term used to globally identify an Info, an Exception or a Failure.
Info:	Info is an occurrence of an Informative condition monitored by the system, which is neither an exception nor a failure.
Exception:	An exception is an occurrence of a condition that is outside of the specified parameters and affecting an outside Line Replaceable Unit (LRU).

Failure:	A failure is an occurrence of a condition that is outside of the specified parameters and affecting this Line Replaceable Unit. A failure normally requires maintenance or replacement to restore affected unit to its normal operation conditions.

1.1.4 Acronyms

ASCII	American Standard Code for Information Interchange
CCITT	Comité Consultatif International Téléphonique et Télégraphique
CDRL	Contract Data Requirements List
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DCM	Door Control Module
DCRP	Door Control Relay Panel
DMS	Door Module System
DPS	Door panel Status
ECD	Engineering Change Document
FLOG	Fault Logger
HW	Hardware
I/O	Input/Output
ICD	Interface Control Document
IEEE	Institute of Electrical and Electronics Engineers
IRAM	Internal Random Access Memory
KB	Kilobytes
LDTS	Local Diagnostic and Test System
LED	Light Emitting Diode
LLC	Limited Liability Corporation
LRU	Line Replaceable Unit
MDC	Master Door Controller
NVRAM	Non-Volatile Random Access Memory
OS	Operating System
PID	Proportional integral Derivative
PTA	Portable Test Application
RAM	Random Access Memory
RDRP	Read Door Real Position
RS232	Recommended Standard 232 (Computer Serial Interface)
RTC	Real-Time Clock
RTDM	Real-Time Data Monitoring
RTS	Request To Send
SCI	Software Configuration Items
SDD	Software Design Description
SRAM	Static Random Access Memory
SRS	Software Requirements Specification
Std	Standard
TL	Trainline
TS	Technical Specification
USB	Universal Serial Bus
VSRS	Vapor Stone Rail Systems

1.1.5 Document Organization

The Software Design Description (SDD) document describes the architectural software design of the DMS Application Real-Time Data Monitoring Module. This SDD is divided into 3 sections and the following specific information is provided within each one:

- Section 1, Introduction, states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.

- Section 2, References, lists the factors that affect the product and its requirements.

- Section 3, Software design summarizes followings:
    - Decomposition: describes functions of software entities.
    - Dependency Description: gives a brief description of the dependency between each entity.
    - Interface Description: describes the interaction between entities.
    - Detailed Design: details the functions implemented by each software entity, presents and explains tasks performed by each software entity in functional conditions.

Within the document, diagrams, flowcharts and tables are used to present the Real-Time Data Monitoring Module software design:

- Decision flowcharts give a graphical view of how entities and functions process work.
- The diagram Figure 3-1 describes the dependencies between different modules.
- Tables contain data structures used by the Real-Time Data Monitoring Module. They also present C code.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, September 23rd 1998, IEEE, www.ieee.org
IEEE Std 1558-2005	IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems, March 23rd 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

S17-07	Software Design Description Template, Rev. 0, 2002-01-15 complemented with requirements from IEEE Std 1558.
201111489	Portable Test Interface Protocol, Interface Control Document (ICD), Rev 0, Date unknown
201111270	Door Control Module Application Software Requirements Specification Document, VSRS, Rev. 01

3 SOFTWARE DESIGN

3.1 Decomposition Description

3.1.1 RTDM Module Description

The Real-Time Data Monitoring Module is a single entity which captures all the relevant signals and puts them in a buffer. After a fault has occurred, a command comes from the Monitoring module to save all the buffered data into the NVRAM and continue to save data until a command is received from the Monitoring module to stop. The Real-Time Data Monitoring Module also manages the 20 snapshot files in the NVRAM.

3.1.2 Concurrent Process Decomposition

3.1.2.1 RTDM Task Description

The LDTS_Task triggers the watchdog, reads the inputs and put them into a ring buffer. After managing the ring buffer, it checks if an existing LDTS snapshot file has expired.
The module functions shall be designed to support multiple concurrent file transfers.

3.1.3 Data Decomposition

The Real-Time Data Monitoring Module receives commands from the Monitoring module. It also manages the 10 snapshots files in the NVRAM.

Structure in Table **3-1** describes the snapshot files array stored in NVRAM.

Table 3-1: Structure of Snapshots Log

Identification	LDTS_Snapshot_File
Entity Type	struct Ldts_SnapshotStruct
Prototype	

```
// Maximum number of LDTS snapshot
#define LDTS_NUMBER         10

// 20 samples before + 20 samples after + fault time (1) ==> 41 records
#define RECORD_NUMBER       41

// Union containing the reading of the inputs
union _Signals
{
    unsigned char ucData[14];
    struct _Ldts_RecordStruct
    {
            unsigned Int  uiSignal0   :1;   // Bit #0
            unsigned Int  uiSignal1   :1;   // Bit #1
            unsigned Int  uiSignal2   :1;   // Bit #1
            unsigned Int  uiSignal3   :1;   // Bit #1
            ...
            unsigned char uiSignal95  :1;   // Bit #95
            Signed Int    iDelta;

    }Ldts_RecordStruct;


// Structure containing the LDTS Snapshot as stored in NVRAM
typedef struct _Ldts_SnapshotStruct
{
    unsigned long   ulTimeStamp;       // Time Stamp of the Snapshot
    unsigned int    uiEventCode;       // Event number that triggered
                                       // the snapshot
    unsigned int    uiValidData;       // Index of first valid record in
                                       // the Ldts_RecordStruct
    unsigned int    uiAssociatedEvent; // Index of associated event in the
                                       // event log file; for cross
                                       // references purposes
    Ldts_RecordStruct Record[RECORD_NUMBER];// Bitmaps Signals recorded
    unsigned int    uiCrc;             // CRC of the records and header
}Ldts_SnapshotStruct;

Ldts_SnapshotStruct huge LDTS_Snapshot_File[LDTS_NUMBER];
```

The snapshot counter structure is used to keep track of state in the recoding process of all 20 snapshot files. When a snapshot command is triggered, the snapshot counter structure is affected. The status of the corresponding snapshot file is activated and the time counter is reloaded to get the next 2 seconds of data.

Table 3-2: Structure of Snapshots Counter

Identification	SnapshotCounter
Entity Type	struct Ldts_SnapshotCounterStruct

Prototype	`// Maximum number of LDTS snapshot`
	`#define LDTS_NUMBER 20`
	
	`typedef struct`
	`{`
	`  // Status of each snapshot`
	`  unsigned long ulStatus;`
	
	`  /* ulStatusReading is usefull for Network, when network is reading,`
	`  corresponding flag to snapshot index`
	`  is set to 1, forcing LDTS.c to use another snaphot array for`
	`  writting. */`
	`  unsigned long ulStatusNetReading;`
	
	`  // Time counter (increments of 250ms)`
	`  unsigned int uiTimeCounter[LDTS_NUMBER];`
	
	`} Ldts_SnapshotCounterStruct;`
	
	`Ldts_SnapshotCounterStruct near SnapshotCounter;`

3.2 Dependency Description

3.2.1 Inter-Module Dependencies

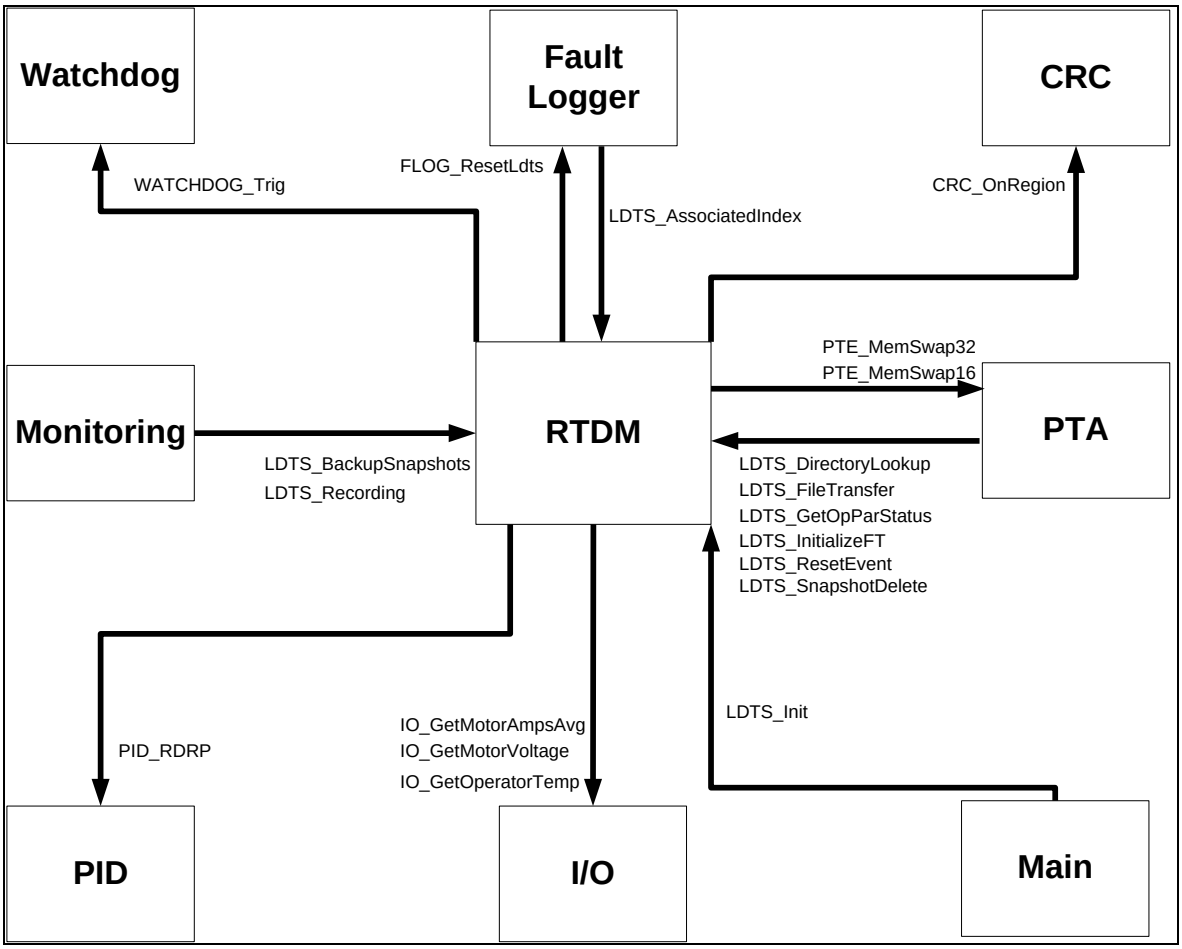

Figure 3-1: Inter-Module Dependencies Diagram

3.2.1.1 Real-Time Data Monitoring Module

Figure 3-1 shows the Inter-Module Dependencies Diagram of the Real-Time Data Monitoring Module.

3.2.2 Inter-Process Dependencies

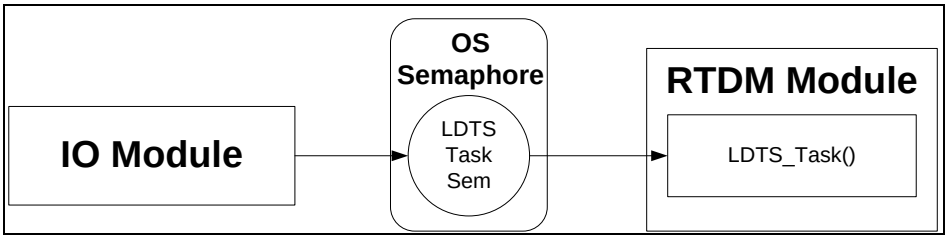

Figure 3-2: Inter-Process Dependencies Diagram

3.2.3 Data Dependencies

RTDM Module uses following from the IO Module:

- IOInput: IO module structure for the inputs of the CPU.
- IO_OutCPU: IO module structure outputs of the CPU.

3.3 Interface Description

This section describes the functions of the Real-Time Data Monitoring Module. The function name, a description of the function, input and output parameters required by the function are provided for each function of each module.

3.3.1 RTDM functions

The RTDM module implements the following functions (sorted alphabetically):

Function Name	unsigned char LDTS_AssociatedIndex (unsigned int uiEventCode)
Description	Returns the Snapshot index if the event code received in parameter is linked to a snapshot.
Input	unsigned int uiEventCode : Event code
Output	None
Return	Snapshot index if it exists, NO_ASSOCIATED_INDEX otherwise

Function Name	void LDTS_BackupSnapshots (void)
Description	Initiate the backups of the snapshots.
Input	None
Output	None
Return	None

Function Name	void LDTS_CheckSnapshot (void)
Description	Verify if an LDTS snapshot is expired
Input	None
Output	None
Return	None

Function Name	void LDTS_CopyData (unsigned char ucSnapshotIndex)
Description	Copy input data into the specified LDTS structure
Input	unsigned char ucSnapshotIndex : Index of the LDTS snapshot to copy
Output	None
Return	None

Function Name	void LDTS_DirectoryLookup (unsigned char ucSnapshotIndex, char *pstrFileName, unsigned long *ulSize
Description	Gives information about a Snapshot.
Input	unsigned char ucSnapshotIndex : Index of the snapshot char *pstrFileName : Pointer to file name unsigned long *ulSize : Pointer to the file size
Output	File Name of the snapshot Size of the snapshot
Return	None

Function Name	unsigned char LDTS_FileTransfer (unsigned char xhuge *pucBuffer, unsigned int *puiBufferSize)
Description	Processes a snapshot file transfer.
Input	unsigned char xhuge *pucBuffer : A pointer to the data destination. unsigned int *puiBufferSize : A pointer to the size of the data.
Output	Contains the data copied Contains the number of bytes copied into the buffer
Return	The function returns: FS_XFER_UNDERWAY while is not completed FS_XFER_OK when the transfer is completed

Function Name	void LDTS_GetOpParStatus(Ldts_RecordStructExt * pLdtsReturnStruct)
Description	Obtain operational parameters.
Input	Ldts_RecordStructExt * pLdtsReturnStruct
Output	Operational parameters structure: Ldts_RecordStructExt * pLdtsReturnStruct
Return	None

Function Name	void LDTS_Init (void)
Description	Initialize the LDTS module variables and create the LDTS_Task
Input	None
Output	None
Return	None

Function Name	unsigned char LDTS_InitializeFT (unsigned char ucSnapshotIndex)
Description	Initializes buffer to get ready for File Transfer.
Input	unsigned char ucSnapshotIndex : Index of the snapshot
Output	None
Return	TRUE if everything is OK, FALSE if there's no snapshot

Function Name	void LDTS_ReadData (void)
Description	Read input data and put it into the ring buffer
Input	None
Output	None
Return	None

Function Name	void LDTS_Recording (unsigned long ulTimeStamp, unsigned int uiEventCode, unsigned int uiAssociatedEvent, unsigned char *pucSnapshotIndex, unsigned int *puiOldSnapshot)
Description	Starts recording a Snapshot.
Input	uiTimeStamp: Time when the fault occurred uiEventCode: Event code of the fault uiAssociatedEv: Associated event of the fault puiSnapIndex: Pointer to the index of the Snapshot file puiOldSnap: Pointer to the old event to be removed
Output	Index of the new snapshot Index of the event removed
Return	None

Function Name	void LDTS_ResetEvent (void)
Description	Resets all events linked to a snapshot.
Input	None
Output	None
Return	None

Function Name	void LDTS_SnapshotDelete (unsigned char ucSnapshotIndex, unsigned int *puiAssociatedEvent)
Description	Deletes data into the specified Snapshot structure.
Input	uiSnapIndex: Index of the snapshot to delete puiAssEv: Pointer to the associated event
Output	Associated event of the Snapshot deleted
Return	None

Function Name	void LDTS_Task (void)

Description	Task that manages the Snapshot.
Input	None
Output	None
Return	None

3.4 Detailed Design

3.4.1 RTDM Module Design

The RTDM module performs the following tasks:

- Snapshot Recording
- Snapshot Delete
- Snapshot Directory Lookup
- Snapshot File Transfer
- Snapshot Reset Event links

3.4.1.1 LDTS_CheckSnapshot

Figure 3-3 shows the sequence used to verify if a snapshot is in progress. If so, data of the expired snapshot is copied and archived. The LDTS Task calls this function.

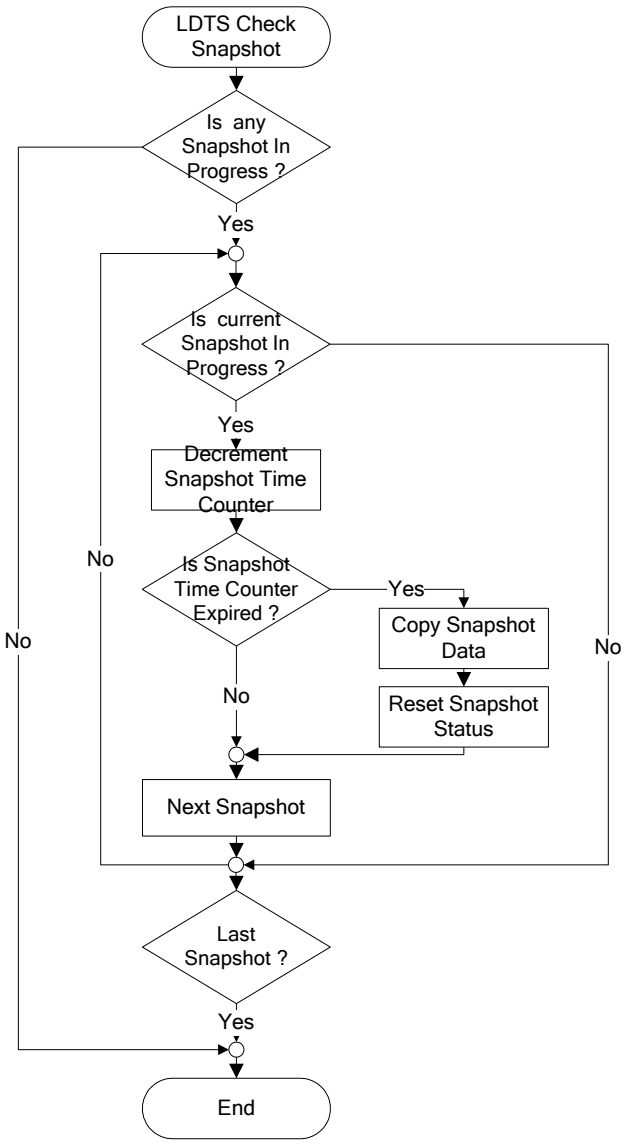

Figure 3-3: LDTS Check Snapshot

3.4.1.2 LDTS_DirectoryLookup

The Snapshot module receives from the PTA a directory lookup command in prevision of a file transfer.
The module receives:

- A pointer to a buffer where the formatted data (DirLookup) is being copied.

The DirLookup structure is described in the ICD document 201111489. The module returns:

- For a Snapshot file, the file name corresponds to the timestamp of the event that triggered the snapshot. The standard “C” type (modified, unsigned 32 bits) of time is copied starting at index 0. The unused bytes are filled with null character (0x00).
- The size of the Snapshot File requested.

In case the Snapshot file is empty, the timestamp returned is 0 and the size is 0.
The module sets the pointers. The index of the first and the last record to be transmitted for the next file transfer request are stored.
To calculate the size of the file, the module simulates the real packet sent to the PTA port. The structure of the message is described in the ICD document 201111489. A filter on data is applied to send only the required signals for Snapshot. All signals are systematically recorded and compressed when the snapshot is triggered and filtering and decompression is needed to comply with the format requested for a network or a PTA transmission.
The module also makes a copy of the Snapshot to be transmitted. It formats the data starting with the header followed by the data. Data are formatted following the structure described in the Portable Test Interface Protocol Interface Control Document (ICD), document # 201111489. Figure 3-4 shows the sequence used to process a Directory Lookup.

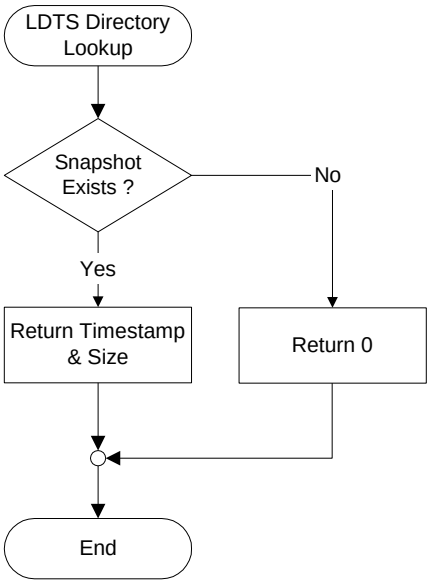

Figure 3-4: Directory Lookup

3.4.2 LDTS_InitializeFT

Snapshot File initialize module initializes data before they are being used and start the first data feed to the sender for the File Transfer.
The module receives:

- A pointer to a buffer where the formatted data is being copied.

The module returns:

- Formatted data in the received buffer.
- Status of the process. (FS_XFER_UNDERWAY, FS_XFER_OK)

The formatted data is copied in the location pointed with the received pointer. The pointer is post incremented and always points to the next available position. The function leaves the pointer at the current position.
The module formats and copies only 6 *32bytes (192 bytes) of data on each request. FS_XFER_OK is returned on the last packet; otherwise, FS_XFER_UNDERWAY is returned with each packet packaged.
Figure 3-5 shows the sequence used to process a File Transfer.

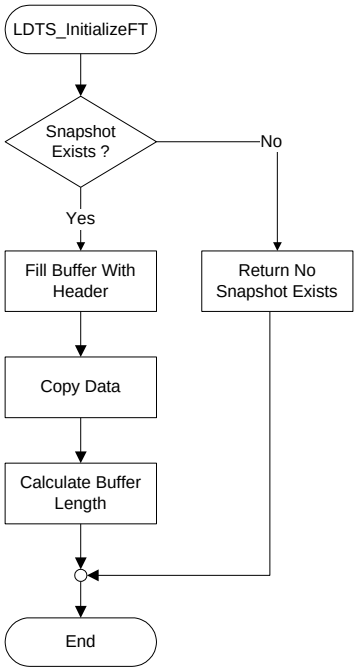

Figure 3-5: LDTS_initializeFT module

3.4.3 LDTS_FileTransfer

Snapshot File transfer module is the module that feeds the data to the sender for the File Transfer. Its functioning is very similar to the LDTS_InitializeFT module.
LDTS_InitializeFT will start the File Transfer and LDTS_FileTransfer will proceed it to the end.

The module receives:

- A pointer to a buffer where the formatted data is being copied.

The module returns:

- Formatted data in the received buffer.
- Status of the process. (FS_XFER_UNDERWAY, FS_XFER_OK)

The formatted data is copied in the location pointed with the received pointer. The pointer is post incremented and always points on the next available position. The function leaves the pointer at the current position.
The module formats and copies only 6 *32bytes (192 bytes) of data on each request. FS_XFER_OK is returned on the last packet; otherwise, FS_XFER_UNDERWAY is returned with each packet packaged.
Figure 3-5 shows the sequence used to process a File Transfer.

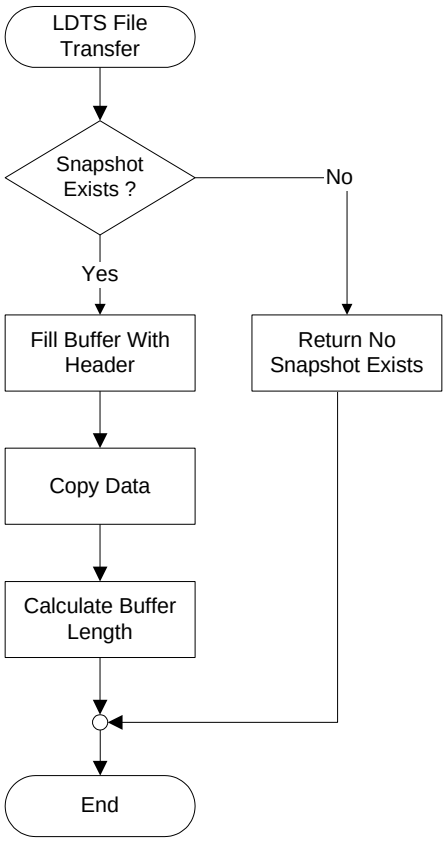

Figure 3-6: LDTS File Transfer

3.4.4 LDTS_Recording

The Monitoring module triggers the snapshot. The 20 samples buffer is a circular buffer that contains the values (6 signals of 2 bytes each and the timestamp of 2 bytes) of the inputs taken at intervals between 50 and 1000ms.

The module receives:

- The event code that triggered the Snapshot.
- The timestamp associated with the event corresponding to the time when the fault occurs.
- The index of the associated event where the event is stored in the Event log file.

The module returns:

- The index of the Snapshot file that will make the links between the Snapshot and the Event in the log file.
- The index of the old event if there was a valid Snapshot file on the current location. The value is used to remove the link between the Snapshot and the Event in the log file of the previous Snapshot file.

Figure 3-7 shows the sequence used to start recording a snapshot.

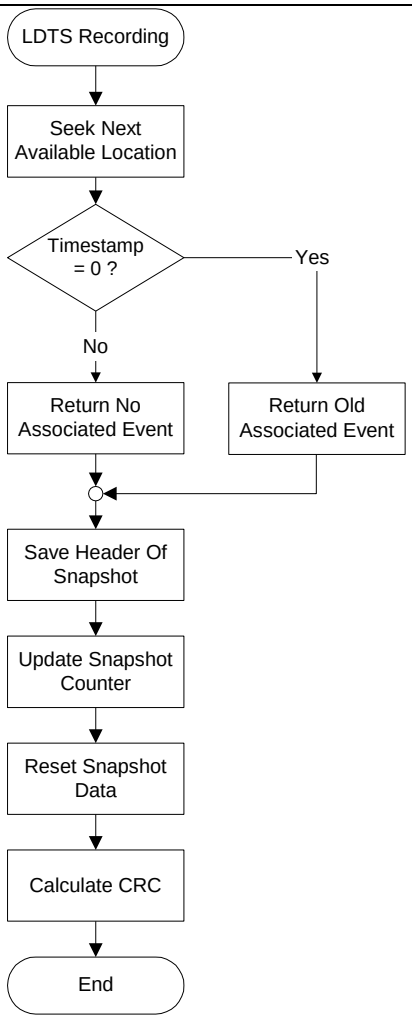

Figure 3-7: LDTS Recording

The sampling period between readings varies between 50ms and 1s. If two 50ms-readings are equal, the new reading will be discarded and a new 50ms timer is started. A next reading will be considered only if readings are not equal (if at least one input, being boolean or analog, changes) or when 1 second after last sample has elapsed.

The first valid reading and beginning of readings pointers are not equal only when a Snapshot is triggered in the first 20 samples after the first execution of the monitoring module. Table 3-3 shows the case when a Snapshot is triggered within the first 20 samples after the monitoring module has started for the first time. The example shows the beginning of readings is at index 0 and the first valid reading is set to index 0x14.

Table 3-3: Structure of Circular Buffer for Snapshot Purposes

	Beginning of Readings				First Valid Readings			
Index #	0x00	0x01	….	0x13	0x14	0x15	….	0x28
Time:	-	-	….	-	-0.5s	-0.25s	….	7.5s
Data:	Null value	Null value	Null value	Null value	Reading	Reading	Readings	Reading

Note: the sampling rate in this table is only an example, since sampling rate can be between 50 ms and 1 second.

The first action taken when a snapshot is triggered is to seek for the next location available to record the current snapshot in the log file. The timestamp of all 10 snapshots records in the log file are read and the oldest snapshot is retained as the next location for the current snapshot.
A validation of the CRC of the current snapshot record is performed. When the snapshot record is valid, the index of event of the old Snapshot is memorized and sent back to the Monitoring module to be re-used by the Fault Logger.
The new header of the snapshot is then recorded in the appropriate LDTS_Snapshot_File structure. The timestamp, the event code, the index of first valid record and the index of the associated event is recorded in the header. The data of the signals is forced to 0 and a CRC on this preliminary snapshot is calculated and inserted.
A counter of 20 samples per second is stored, the pointer of the beginning of data in the circular buffer is stored and the status bit is updated. These data are stored into the Snapshot counter.

The Ldts_SnapshotCounterStruct structure (see Table 3-2) contains information for 20 Snapshot Files:

- A 16-bit value represents the status of the 10 first snapshots. The least significant bit corresponds to the status of snapshot 1 and bit #9 is the status of snapshot 10. A TRUE bit means that the recording of a snapshot is in progress and the counter associated is to be serviced. A status bit set to FALSE means that there is no recording in progress and it is available for a new snapshot.
- The counter remaining before the end of 20 samples reached.
- The 10 last snapshots (out of 20) are used to store a copy of the 10 first snapshots when the Manual lock is activated (Cutout). These snapshots cannot be recorded in real-time.

When the counter of an active snapshot reaches 0, the associated status bit is set to FALSE, the 20 readings data recorded is copied from the circular buffer into the associated Snapshot File.
Ten Snapshots can be started at the same time. In case another Snapshot triggered while there are already 10 in progress, the one with the lowest time counter is replaced.

Each reading in the circular buffer of the 20 snapshots contains the LDTS_Snapshot_File structure shown in Table 3-1.

- 4 bytes for the timestamp
- 2 bytes for the event code (fault number)
- 1 bytes for the index of the first record that contain valid data
- 1 byte for indication that the structure is being modified
- 2 bytes for the index of the associated Event in the Event log file.
- Record of parameters = 20 records before fault + Record of the fault + 20 records after fault => 41 records of:
    - 14 bytes for signals
    - 2 bytes for delta time
- 2 bytes for the CRC

The Timestamp is the number of seconds elapse since January 1st, 1970.
The Event code is the fault number of the fault that caused the snapshot.
The Index of the first valid record is the index of the first valid record in case there is not enough data recorded before the event happens such as when there is less than 1 second of data in memory. The invalid data is padded with null values and the index is adjusted to point on the first valid record.
The Index of Event Log is used to point to the event in the Event Log file that has triggered the snapshot. This index is only used locally. The index is used to remove the link in the associated Event in the Event log File in case the Snapshot File is deleted or replaced by a new one. There is only a maximum of 20 Events in the Fault Log File that point to the 20 Snapshot files.
Signal bit 0, signal bit 1, signal bit 2, signal bit 3, signal bit 4, signal bit 5 and signal bit 6 are the status of the operating parameters of the DCM.
Delta Time is the time elapsed since the last reading.
The CRC is calculated according to the CRC-CCITT Standard.

The following calculates the amount of memory space required for the snapshots.
41 x (14 bytes) = 574 bytes
574 bytes + 10 bytes for header + 2 bytes for the CRC = 586 bytes
(20 buffers for Snapshot) x 586 bytes = 11720 bytes

Each snapshot contains between 2 and 40 seconds of data (20 samples before and 20 samples after the fault with a sampling period of 50ms to 1s.).
Table 3-4 lists the signals used in a typical snapshot. Each bit represents a signal. All bits are regrouped together in 14 bytes and is named a “sampling”. These 14 bytes are sampled every 50ms to 1s.
Ten snapshots are recorded in a circular buffer fashion; therefore, a new file overwrites the oldest one. Upon activation of the Manual Lock, those 10 snapshots are copied in the snapshot locations 11 to 20 to preserve them for deferred troubleshooting.

Table 3-4: Signals List Example

#	Signal Name	Related To Fault	Signal Source
1	Open Car Line	All	Input Sense Point
2	Unlock Car Line	All	Input Sense Point
3	Motor Powered	All	Input Sense Point
4	Emergency Handle Activated	All	Input Sense Point
5	Mechanical Lock in "Normal"	All	Input Sense Point
6	Panel not Closed	All	Input Sense Point
7	Panel Unlocked 1	All	Input Sense Point
8	Encoder Phase A	All	Input Sense Point
9	Encoder Phase B	All	Input Sense Point
10	Motor Temperature High	All	Input Sense Point
11	Motor Temperature Valid	All	Input Sense Point
12	Open Interlock Powered	All	Input Sense Point
13	Output Door Warning LED	All	Output

#	Signal Name	Related To Fault	Signal Source
14	Output Negative Spare Activation	All	Output
15	Output Negative Spare Feedback	All	Output
16	Output Negative Spare Other Side	All	Output
17	Motor Current	All	Input Sense Point
18	5V Logic Supply	All	Input Sense Point
19	5V Encoder Supply	All	Input Sense Point
20	5V Shared Supply	All	Input Sense Point
21	15V Motor Supply	All	Input Sense Point
22	MDC Car Uncoupled	All	Input Sense Point (MDC)
23	Network Available	All	CAR NETWORK Input
24	MDC Partial Close Right	All	Input Sense Point (MDC)
25	MDC Partial Close Left	All	Input Sense Point (MDC)
26	MDC Local Recycle	All	Input Sense Point (MDC)
27	MDC Door Open Right	All	Input Sense Point (MDC)
28	MDC Door Open Left	All	Input Sense Point (MDC)
29	MDC Zone Light	All	Input Sense Point (MDC)
30	DCRP Door Open TL	All	Input Sense Point (DCRP)
31	DCRP Door Unlock TL	All	Input Sense Point (DCRP)
32	DCRP Crew Switch Relay High	All	Input Sense Point (DCRP)
33	DCRP Crew Switch Relay Low	All	Input Sense Point (DCRP)
34	DCRP Signal Light Relay Drive	All	Input Sense Point (DCRP)
35	DCRP Signal Light Relay Contact	All	Input Sense Point (DCRP)
36	DCRP Door Enable TL	All	Input Sense Point (DCRP)
37	Output Door Failure LED	All	Output
38	Output Door Normal LED	All	Output
39	Output Solenoid Driven Activation	All	Output
40	Output Solenoid Driven Feedback	All	Output
41	Output Motor Driven	All	Output
42	Output Closing Light Activation	All	Output
43	Output Closing Light Feedback	All	Output
44	Output Closing Light Other Side	All	Output
45	Output Positive Spare Activation	All	Output
46	Output Positive Spare Feedback	All	Output
47	Output Positive Spare Other Side	All	Output
48	Door Position (inches)	All	Input Sense Point
49	5V Shared Supply Overload	All	Input Sense Point
50	Motor Overcurrent Shutdown Activated	All	Input Sense Point
51	Panel Unlocked 2	All	Input Sense Point
52	Encoder Index	All	Input Sense Point
53	V-Gate Open Supply	All	Input Sense Point
54	V-Gate Close Supply	All	Input Sense Point
55	Solenoid Lock Interlock Powered	All	Input Sense Point
56	Negative Spare Input	All	Input Sense Point
57	Positive Spare Input	All	Input Sense Point
58	MDC Zone Line	All	Input Sense Point (MDC)
59	DPS (Door Panel Status Chain)	All	Input Sense Point
60	DCRP Door Enable Relay Contact	All	Input Sense Point (DCRP)

3.4.5 LDTS_ResetEvent

When the Event Log file is deleted, the links in the Snapshot file is to be removed to ensure that a File does not point anymore on a deleted Event record. The module resets the “Index of the Associated Event” of all valid Snapshot Files and recalculates the new CRC of the file.
No parameters are receives or return for that function.
Figure 3-8 shows the sequence used to process and LDTS Reset Event.

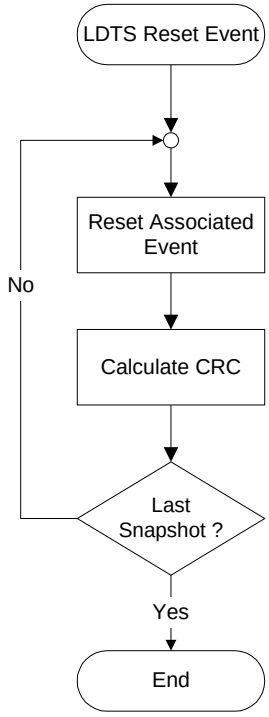

Figure 3-8: LDTS Reset Event

3.4.6 LDTS_SnapshotDelete

The Snapshot module receives from the Monitoring module the command to delete individually the Snapshot. The deletion of a Snapshot file is achieved by writing 0x00 in the timestamp field of the file. The module receives the index of the file to be deleted as parameter and returns to the Monitoring module the index of the associated event in the even log file to be reset.

Figure 3-9 shows the sequence used to process a Snapshot Deletion.

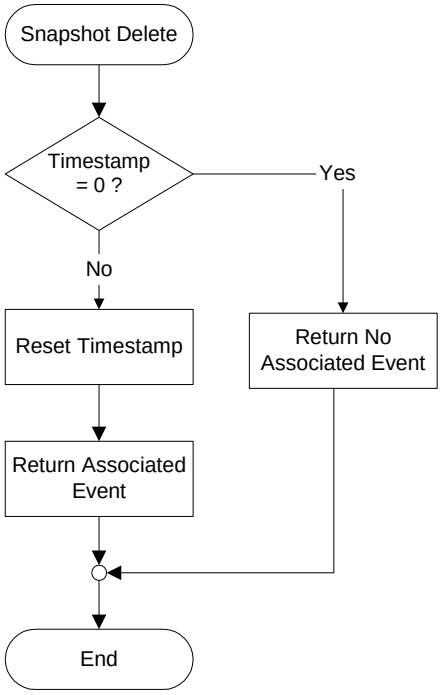

Figure 3-9: Snapshot Deletion

3.4.6.1 LDTS_Task

Figure 3-10 shows the task used to record the data for each snapshot triggered. This task is scheduled to run once every 50ms.

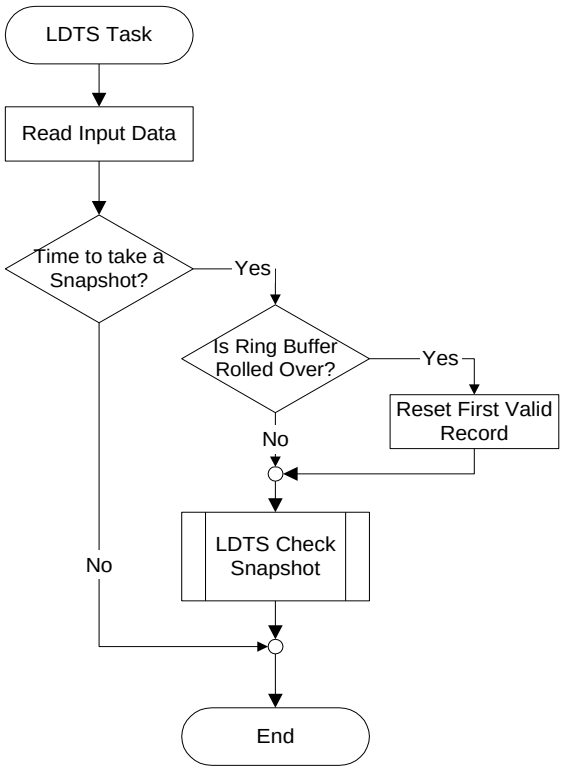

Figure 3-10: LDTS Task

3.4.7 Data Entities Detailed Design

Not applicable.